I have done two projects here(not in production) -- 1/ Data migration app(exe file),in which it reads csv files from a folder in particular module(like sales,policy etc) and make a temporary table in sql server database and moved the csv files to archive folder when whole data is migrated to sql server db.2/ RAG chatbot (document based) in dotnet using foundry local phi4 llm model.Now i am working on ai based regression testing app.

These three projects are a strong foundation—you are already building exactly the skills that make an AI engineer valuable in reinsurance: **data reliability, document AI, and AI quality assurance**. The key now is to upgrade them from "working prototypes" into well-evaluated, production-minded products.

What each project signals

Project	Strong signal	Upgrade it into
CSV-to-SQL Server migration app	Data engineering, file handling, SQL, workflow automation	A governed data-ingestion and validation pipeline
.NET RAG chatbot with Phi-4	LLM application development, local inference, document retrieval	An enterprise document assistant with evaluation and guardrails
AI-based regression testing app	AI quality engineering, automation, reliability	An AI test/evaluation platform for both traditional and LLM systems

Together, these show you can work across the AI lifecycle: ingest data, build an AI application, and test whether it behaves reliably. That is more valuable than only knowing how to train a model. [1] [2]

Improve the migration app

Your app currently moves CSVs into temporary SQL Server tables and archives them after loading. To make it more impressive, add **validation, reconciliation, and observability**.

Add these capabilities:

- **Schema validation:** Check file names, columns, data types, nullability, and row counts before loading.

- **Business-rule validation:** Flag invalid dates, negative amounts, duplicate keys, missing mandatory fields, and unexpected category values.
- **Reconciliation checks:** Compare source CSV row counts and totals with SQL Server counts and aggregates; use checksums or row hashes for critical tables. [3] [4]
- **Error handling:** Send rejected rows to a quarantine table or error file with a reason code; never silently skip records.
- **Audit trail:** Record file name, batch ID, load start/end time, rows read, rows loaded, rows rejected, and validation results.
- **Idempotency:** Ensure rerunning the same file does not create duplicate records.
- **Rollback or recovery:** If a batch fails midway, define whether it retries, resumes, or rolls back cleanly.
- **Dashboard/report:** Generate a simple migration report after each batch.

A strong résumé line could be:

> Built a CSV-to-SQL Server data-migration application with schema validation, batch-level reconciliation, rejected-record handling, audit logging, and automated archiving.

Microsoft's migration guidance specifically emphasizes validation queries against source and target, isolated test environments, and post-migration checks for accuracy, completeness, and performance. [5] [3]

Improve the RAG chatbot

A local Phi-4 RAG chatbot is good, but interviewers and business stakeholders will ask: "**How do you know its answers are correct?**" Build the answer into the product.

Add:

- **Source citations:** Every answer should show the document name, page, and quoted passage used.
- **Retrieval evaluation:** Measure whether the correct chunk was retrieved, not only whether the final answer sounds good.
- **Answer evaluation:** Create 30–100 question–answer pairs with expected sources and key facts; test accuracy, hallucination, refusal behavior, and completeness. [2] [1]
- **Guardrails:** Block questions outside scope, detect prompt injection, prevent disclosure of sensitive data, and add a "not found in documents" response.
- **Confidence/uncertainty handling:** If retrieval similarity is low or sources conflict, route the question to a human.
- **Feedback loop:** Let users mark answers helpful/unhelpful and store those cases as future regression tests.
- **Better architecture:** Separate ingestion, chunking, embedding, retrieval, reranking, generation, and evaluation into clear modules.

- **Deployment path:** Containerize the service, add logging, latency tracking, and a simple API/UI.

A strong résumé line could be:

> Developed a .NET document-RAG assistant using a locally hosted Phi-4 model, with source-grounded responses, retrieval evaluation, guardrails, and a feedback-driven regression test set.

For enterprise RAG, evaluation frameworks, representative ground-truth datasets, automated checks, human review, and guardrails are considered essential rather than optional. [1]

Make the regression-testing app your flagship

This is your best opportunity because it connects directly to AI reliability—an area many AI engineers neglect. AI regression testing is not just "run old tests again"; it means checking that changes to prompts, models, retrieval indexes, tools, or workflows do not silently break earlier behavior. [6] [2]

Build it in layers:

1. **Deterministic checks:** JSON schema validation, required fields, regex checks, response-time limits, API status codes.
2. **Task-specific checks:** Does the answer contain the required fact? Is the cited source correct? Does the extracted field match the expected value?
3. **LLM-as-judge checks:** Score helpfulness, factual consistency, completeness, tone, and safety using a rubric.
4. **Human review:** Keep humans in the loop for high-risk or ambiguous cases. [2] [1]
5. **Regression suite:** Store every fixed bug or production failure as a repeatable test case.
6. **Comparison reports:** Compare the current model/prompt version with the previous baseline; show pass rate, failure rate, latency, cost, and changed failures.
7. **CI/CD integration:** Run fast smoke tests on every code or prompt change, and run the full evaluation suite before release. [6]

A strong résumé line could be:

> Building an AI regression-testing platform that evaluates LLM outputs using deterministic checks, rubric-based LLM-as-judge scoring, source-grounding validation, and baseline comparison across prompt/model changes.

Skills to add next

Your current stack appears to be .NET, SQL Server, local LLMs, and AI testing. To become more marketable, add:

- **Python:** Even if your company uses .NET, Python is essential for broader AI/ML roles.
- **Cloud AI:** Azure OpenAI, Azure AI Search, Azure Functions, Azure Container Apps, or AWS equivalents.

- **Vector databases:** Azure AI Search, Qdrant, Pinecone, Weaviate, or pgvector.
- **RAG evaluation:** Ragas, DeepEval, Promptfoo, LangSmith, or a custom evaluation harness.
- **MLOps/LLMOps:** Docker, GitHub Actions, model/version tracking, monitoring, and automated evaluation.
- **Classical ML:** XGBoost/LightGBM, feature engineering, and explainability for insurance risk use cases.
- **Testing tools:** Playwright or Selenium for UI regression, plus pytest/NUnit for code-level tests.

LLM testing should combine code checks, evaluations, trace analysis, performance checks, and human review; production failures should become regression tests so they do not recur. [2]

How to present your experience

Do not describe yourself as someone who "built a chatbot" or "made a migration tool." Present yourself as an engineer who builds **trustworthy AI systems**.

Use this positioning:

> "I build and validate AI-powered workflow automation for reinsurance. My work spans document-based RAG systems, data migration and validation, and AI regression testing. I focus on making AI usable in real business processes—accurate, traceable, testable, and safe to deploy."

That is a much stronger story than "I used YOLO before and now I use LLMs." Your computer-vision background remains useful, especially if you later work on document-image extraction, claims-photo analysis, fraud detection, or visual regression testing. [7]

Next 60–90 days

- Add validation and audit reporting to the migration app.
- Add citations, evaluation, and guardrails to the RAG chatbot.
- Build a small evaluation dataset for the RAG bot and connect it to your regression-testing app.
- Write a one-page internal proposal showing time saved, error reduction, or quality improvement.
- Create a public portfolio version using synthetic documents and data, since company data likely cannot be shared.
- Prepare a 5-minute demo for each project: problem, architecture, evaluation, limitations, and business value.

Your projects are already relevant. The next step is to make them **measurable, governed, and production-ready**—that is what will distinguish you in AI roles inside insurance, reinsurance, banking, and other regulated industries. [1] [2]

⁂

1. https://arxiv.org/html/2407.07858v1
2. https://www.langchain.com/resources/llm-testing
3. https://learn.microsoft.com/en-us/data-migration/sql-server/database/guide
4. https://kanerika.com/blogs/data-migration-testing/
5. https://learn.microsoft.com/en-us/sql/relational-databases/post-migration-validation-and-optimization-guide?view=sql-server-ver17
6. https://pathubs.com/blog/llm-regression-testing
7. https://cloudqa.io/ai-in-software-testing-guide/
8. https://learn.microsoft.com/en-in/ssms/migrate/migrate-sql-server-azure-sql
9. https://www.datafold.com/blog/data-migration-testing-strategy/
10. https://techcommunity.microsoft.com/blog/modernizationbestpracticesblog/data-migration-strategies-for-large-scale-sybase-to-sql-migrations-using-ssma-ss/4471308
11. https://www.confident-ai.com/blog/llm-testing-in-2024-top-methods-and-strategies?3a0b03e1_page=11
12. https://www.compilenrun.com/docs/language/sql/sql-data-migration/sql-migration-testing/
13. https://www.vervali.com/in/blog/ai-and-llm-application-testing-in-2026-the-definitive-guide/
14. https://learn.microsoft.com/da-dk/data-migration/sql-server/virtual-machines/guide?view=azuresql-vm
15. https://skyvia.com/blog/how-to-connect-sql-server-to-sql-server/